Characterizing CUDA Unified Memory (UM)-Aware MPI Designs on Modern GPU Architectures

K. V. Manian, A. A. Ammar, A. Ruhela, C.-H. Chu, H. Subramoni and D. K. Panda
The Ohio State University
Columbus, Ohio
{vadambacherimanian.1,awan.10,ruhela.2,chu.368,subramoni.1,panda.2}@osu.edu

ABSTRACT

The CUDA Unified Memory (UM) interface enables a significantly simpler programming paradigm and has the potential to fundamentally change the way programmers write CUDA applications in the future. Although UM leads to high productivity in programming using CUDA by simplifying the programmer's view of CPU and GPU memory spaces, initial support for UM in the Kepler series of GPUs lacked in performance necessitating several UM-aware designs in state-of-the-art MPI runtimes such as MVAPICH2-GDR. This has enabled end MPI applications to take advantage of the high productivity promised by UM along with high performance. However, as CUDA runtimes and GPU architectures advance, the performance offered by UM has also improved significantly. Thus, there is a need to re-evaluate the performance characteristics of UM in light of these changes to understand how the UM-aware designs in state-of-the-art MPI runtimes must be adapted. We take up this broad challenge and characterize the performance of UM-aware MPI operations and to gain insights on how MPI runtimes need to deal with UM-based data residing on GPU and CPU for different generations of GPU architectures. Our characterization studies show that UM designs conceived during the Kepler GPU era still stands valid and provide valuable performance improvement on the latest Pascal and Volta GPUs. Furthermore, performance evaluation of optimized UM designs show that they outperform naive designs on MVAPICH2-GDR and Open MPI by 4.2× and 2.8× respectively for Intel systems. Additionally, the DD experiments for pure device transfers also show that MVAPICH2-GDR is up to 12.6× better than OpenMPI (w/ UCX).

KEYWORDS

CUDA, Unified Memory, MPI, HPC

1 INTRODUCTION

Recent advances in Graphics Processing Unit (GPU) architectures have significantly improved the performance of classic as well as cutting-edge High Performance Computing (HPC) applications [1, 5, 8, 17, 32]. Primarily, the introduction of the compute unified device architecture (CUDA) [26] by NVIDIA has led to enhanced programmer productivity. Several versions of CUDA have been released over the last decade enabling application programmers to develop GPU-based applications with high performance and high productivity. One of the very recent developments in CUDA is the Unified Memory (UM) interface. UM enables a significantly simpler programming paradigm and has the potential to fundamentally change the way programmers write CUDA applications in the future. CUDA UM APIs, including *cudaMallocManaged* with hints and prefetching interfaces like *cudaMemAdvise* and *cudaMemPrefetch*, coupled with latest Pascal/Volta GPU architectures enable a newer and simpler program design that is not only highly productive but also provides high performance. Furthermore, the significantly higher memory bandwidth and GPU page-fault engines on Pascal/Volta GPUs seems to push this even further. The key idea is that programmers no longer need to think about GPU and CPU memory as two distinct memory spaces; instead, the GPU, as well as the CPU memory, now form a single unified memory pool and *cudaMallocManaged* is the only call needed to allocate memory from this pool.

On the other hand, there has been a steady increase in the adoption of GPUs by the HPC community. Several applications have been ported to GPUs by using CUDA to program the most compute-intensive parts and utilize other programming models like the *de facto* Message Passing Interface (MPI) to handle communication within and across nodes of a GPU cluster. Before the introduction of CUDA features like Unified Virtual Addressing (UVA) [27] and

*This research is supported in part by NSF grants #CNS-1513120, #ACI-1450440, #CCF-1565414, and NSF ACI 1664137.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
GPGPU-12, April 13, 2019, Providence, RI, USA
© 2019 Association for Computing Machinery.
ACM ISBN 978-1-4503-6255-9/19/04…$15.00
https://doi.org/10.1145/3300053.3319419

GPUDirect [23], MPI+CUDA programs were extremely complex because programmers had to understand two different programming models and carefully optimize the interaction of memory management and data movement performed by CUDA as well as MPI. However, with the advent of UVA and GPUDirect technology such as Inter-Process Communication (IPC) [30], and GPUDirect Remote Direct Memory Access (RDMA) [28], CUDA-Aware MPI has become the new de facto for expressing MPI+CUDA applications. To this end, many research studies [4, 29, 30] have focused on developing efficient support in CUDA-Aware MPI runtimes like MVAPICH2-GDR [35]. Along with this direction, there are developments for direct usage of Unified Memory (UM) buffers in MPI operations [7, 11]. However, with ever evolving GPU architectures and enhanced software support in recent years, the unified memory support needs to be revisited in a systematic fashion.

Current CUDA-Aware MPI libraries offer Unified Memory support. However, MPI+CUDA applications have not been updated with such UM style allocations. Partly, because Kepler series GPUs provided significantly lower performance for UM compared to explicit GPU-based buffers (allocated via cudaMalloc), and more so by a lack of recent benchmarking studies that offer insights about much-improved performance of Unified Memory on Pascal and Volta GPUs.

1.1 Motivation and Challenges

Henceforth, the main motivation of this study is to investigate and revisit data movement designs in state-of-the-art MPI runtimes that are affected by ***advances in architectural support for CUDA Unified Memory features***. These include advanced GPU page fault engines available on Pascal and Volta GPUs as well as *cudaMemPrefetch* and *cudaMemAdvise* APIs available in CUDA 9 and 10 that offer fine-grained optimizations to achieve near device-level (cudaMalloc'ed buffers) performance. It is also pertinent to mention that this research is in part motivated by inefficient unified memory performance of the older generation K-80 GPUs and our earlier designs that addressed performance degradation for such systems [7, 11] that seem to be no longer necessary in the wake of efficient and fine-grained GPU page faults. To address the lack of insights about CUDA UM performance in the context of MPI operations, the following broad challenge is investigated: ***How can CUDA-Aware MPI runtimes deal with diverse GPU architectures and maintain efficient and high-performance designs for unified memory data movement across GPU generations?***. This broad challenge leads to the following key questions that need to be addressed:

- What is the fundamental differentiating factor on newer GPUs that did not exist on previous generation GPUs like K-80?
- Can the performance of Unified-Memory-Aware MPI operations be characterized such that insights can be gained for both cases of unified memory allocation (data residing on GPU and CPU)?
- How advances in Unified Memory need to be dealt with in the context of MPI communication on systems with different GPU generations?

1.2 Contributions

To the best of our knowledge, little exists in state-of-the-art literature that offers insights into the unified memory performance of modern GPU architectures like Pascal/Volta and CUDA-Aware MPI runtimes. The following are the key contributions of this paper:

- Provide an in-depth analysis of CUDA Unified Memory features and designs including a brief history as well as insights into advances in the GPU hardware and CUDA programming model
- Characterize the performance of unified memory in the context of MPI-based intra-node communication on three GPU architectures (K-80, P100, and V100)
- Evaluate performance of Unified Memory MPI operations for the two distinct cases of data residing on the GPU's memory and the CPU's memory by explicitly calling a CUDA kernel before and after MPI calls

The rest of the paper is organized as follows. Section II provides the background knowledge about CUDA, Unified Memory and CUDA-Aware MPI. Section III discusses the state of the art UM designs implemented in MVAPICH2-GDR library. Section IV discusses the evaluation platforms, experimental setup and the results from performance evaluation and characterization. Section V discusses the related works while Section VI concludes this work.

2 BACKGROUND

In this section, we provide the necessary background knowledge related to this paper.

2.1 NVIDIA GPU, CUDA and Unified Memory Technology

NVIDIA General Purpose Graphics Processing Unit (GPGPU) has been widely deployed in HPC systems to accelerate applications due to its massive parallelism and high memory bandwidth. Compute Unified Device Architecture (CUDA) is the parallel computing platform and programming model to harness the computing power of NVIDIA GPUs. Unified Virtual Addressing (UVA), which was introduced in CUDA 4.0, provides a single virtual memory address space for both CPU and GPU memory. UVA allows a pointer to be accessed from GPU regardless of its physical location, i.e., system or GPU memory. **Unified memory** was first introduced in CUDA 6.0 release for NVIDIA Kepler GPU architecture to provide a

single, unified virtual address space to applications for accessing CPU and GPU memory. The managed memory is shared between CPU and GPU, and the CUDA driver automatically migrates data between CPU and GPU when needed instead of explicit data transfer in applications.

Since the release of CUDA 8.0 and NVIDIA Pascal GPU architecture, the hardware support for unified memory has been significantly improved [24]. Memory page faulting support is a significant enhancement to enable on-demand migration and concurrent access. With on-demand migration, performance improvement can be expected because of a small amount of page movement and better data locality. In the latest NVIDIA Volta GPU architecture, the access counter hardware feature helps to optimize the performance of unified memory further by ensuring memory pages are moved to the physical memory of the processor, i.e., to the physical memory of CPU or GPU whichever is accessing the pages most frequently [25]. Moreover, with Volta over NVLink2 interconnect, address translation services (ATS) allows the GPU to directly access page tables of CPU, which can further improve the productivity for developers [33].

2.2 CUDA-Aware MPI

Message Passing Interface (MPI) is a de-facto standard programming model used for parallel applications on HPC systems. With the popularity of GPU-enabled HPC systems in recent years, GPU-Aware MPI (or CUDA-Aware MPI specifically for NVIDIA GPUs) has been introduced [35] and widely adopted in the communication libraries such as OpenMPI [10], MVAPICH2 [22], and more [13]. GPU-Aware MPI allows developers to pass pointers of GPU memory to MPI APIs without explicitly handling the data movement between CPU and GPU at application-level. Advanced CUDA-Aware MPI libraries take advantage of NVIDIA GPUDirect technology such as Peer-to-Peer (P2P) Communication and GPUDirect Remote Direct Memory Access (RDMA) [23] to provide low-latency and high-bandwidth data transfer among GPUs within and across nodes. CUDA managed memory-aware MPI designs [7, 11] are proposed and evaluated on NVIDIA Kepler GPU architecture. As mentioned in Section 2.1, unified memory technology has been significantly evolved over the years. Therefore, there is a dearth of understanding of CUDA-Aware MPI with the use of CUDA Unified Memory on the modern GPU architectures like NVIDIA Pascal and Volta. This paper aims to fill some of this gap in knowledge.

3 ANALYSIS OF STATE-OF-THE-ART UNIFIED MEMORY MPI DESIGNS

An analysis is first done on how MPI runtimes have evolved in the context of support for performing MPI operations like MPI_Send and MPI_Recv from Unified Memory buffers. The following sections discuss the various levels of UM support available in MPI runtimes.

3.1 Basic Unified Memory Support

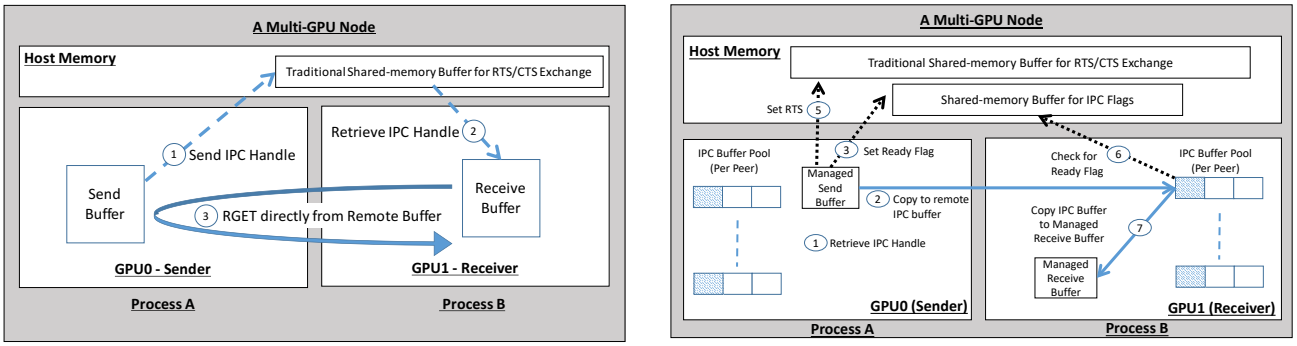
Broadly, naive support to perform such data transfer operations from managed memory to managed memory is available by default with all MPI runtimes irrespective of whether they are CUDA-aware or not. In the naive design, the MPI runtime considers the managed memory buffer as a host buffer and relies on the CUDA runtime to transparently move the data from the device to the host on the sender side on initiation of the data transfer. Once the data has arrived on the host at the sender side, the MPI runtime is then free to perform the transfer from the local process to the remote process. The data would reside on the host at the receiver side until a kernel on the GPU access the data. When such access occurs, the driver transfers the data from the host to the device. As one can see, although this simplifies the design of the MPI runtime considerably, it presents significant performance bottlenecks due to a) the inability to pin managed memory buffers in the Kepler series of GPUs and b) the increased kernel launch time caused due the requirement to transfer the data from the host to the device on the receiver end.

3.2 Advanced Unified Memory Support

Figure 1 shows some of the earlier designs proposed by authors in [11] for advanced UM support in MPI by exploiting features like CUDA IPC and GDR to offer better performance on the K-80 GPUs. However, with a change in GPU hardware as well as CUDA software, we need to enhance both the understanding, and implications of the designs made for a much slower GPU hardware and legacy CUDA interface.

Figure 1(a) depicts the current RGET-based direct IPC design being used by MVAPICH2-GDR. Initially, the sender maps the user GPU buffer and sends the IPC memory handle to the receiver. The receiver performs a direct cudaMemcpy from the source user buffer to destination user buffer and sends the FIN message to the sender completing the transfer. MVPACH2-GDR also uses an internal caching mechanism to amortize the IPC handle creation overhead.

However, since the use of UM in the Kepler series of GPUs negated the ability to pin buffers, one could not use the traditional IPC mechanism depicted in Figure 1(a) to transfer data from device-to-device within a node. To circumvent this limitation, authors in [11] proposed an advanced design depicted in Figure 1(b). As depicted in the figure, the MPI runtime creates a pre-registered pool of IPC buffers at the sender and receiver sides whose IPC handles as well as locations are known ahead of time to the sender and the receiver. Once the managed memory based transfer is initiated by the application, the MPI runtime would copy the data from the managed memory buffer on the device to the pre-registered



(a) Direct IPC-based Design for Device buffers– Courtesy [11]

(b) Advanced IPC Design for Unified Memory buffers (MV2-OPT)– Courtesy [11]

Figure 1: Advanced designs for Device and Unified Memory in MVAPICH2-GDR

IPC buffer on the sender side. The sender would then initiate a control message indicating the location of the pre-registered IPC buffer containing the data. The receiver on getting this control message will proceed to perform a direct cudaMemcpy from the pre-registered buffer on the sender side to the pre-registered buffer on the receiver side. Once this transfer is complete, the data is then copied to the managed memory buffer on the receiver side, and a FIN message is sent to the host indicating the completion of the data transfer.

4 PERFORMANCE EVALUATION AND CHARACTERIZATION

This section discusses, at length, the experiments performed to characterize managed memory performance of MPI runtimes on K-80, P100, and V100 GPUs. First, the evaluation platforms are discussed followed by a discussion on the benchmarks used for this characterization. Subsequent sections contain results on various benchmark/platform combinations.

4.1 Evaluation Platforms

Our experiments were performed on a local cluster that has nodes with three generations of GPUs: 1) Kepler K-80 (hosted on Intel Broadwell), 2) Pascal P100 (hosted on Intel Haswell), and 3) Volta V100 (hosted on Intel Haswell and IBM OpenPOWER). The details of the various host systems are given as follows.

The Broadwell node was using Intel Haswell E5 v2680 processors running at 2.4 GHz. Each node had two sockets with 14 cores per socket connected using Intel QPI. The nodes run Red Hat Enterprise Linux Server release 7.4 (Core) with a kernel version of 3.10.0-693.17.1. Mellanox OFED 4.2-1.2 was used.

The Haswell node was using Intel Haswell E5-2687W processors running at 3.1 GHz. Each node had two sockets with ten cores per socket connected using Intel QPI. The nodes run Red Hat Enterprise Linux Server release 7.4 (Core) with a kernel version of 3.10.0-693.21.1. Mellanox OFED 4.4-2.0.7 was used.

Each node in the OpenPOWER cluster has two POWER9 CPUs, each connected to two GPUs each using NVLink2. Each socket has 22 IBM Power9 cores with four hardware threads per core. The two CPU sockets are connected using X-bus [36] interconnect. The nodes run Red Hat Enterprise Linux Server release 7.5 (Maipo) with a kernel version of 4.14.0-49.18.1. Mellanox OFED 4.3-4 was used.

4.2 Experimental Setup

This section discusses the various components of the experimental setup needed for characterizing the Unified Memory performance of MPI communication operations.

The experiments were conducted by utilizing the OSU Micro Benchmark (OMB) [21] suite which has a collection of benchmarks for analyzing point to point, collective and one-sided MPI operations. More specifically osu_latency benchmark is chosen from the OMB suite for characterizing the unified memory. Since the current osu_latency benchmark sets the effective location of the unified memory buffer as host buffer, insights into data movement performed by the CUDA driver for UM buffers cannot be extracted. Thus, the osu_latency benchmark is modified to move the unified memory buffer to the device using a sender-side CUDA kernel for proper characterization of unified memory performance when the effective location of the UM buffer is the GPU's memory. The original and modified version of osu_latency benchmark is referred to as OMB and MOMB, respectively. Since the goal of the paper is to characterize UM on various GPU systems, OSU Microbenchmarks is used instead of real-world applications. Microbenchmarks provide more flexibility in characterizing a system than a full-fledged application.

Experiments were conducted to evaluate both MVAPICH2-GDR and OpenMPI (built with UCX 1.4 CUDA support) for three combinations: 1) Host to Host transfer *(HH)*, 2) Device

to Device transfer *(DD)*, and 3) Unified (Managed) to Unified (Managed) buffer transfer *(MM)*. On a POWER system, SpectrumMPI is also evaluated. The results from these experiments help us characterize Unified Memory performance of MPI runtimes and provide valuable insights.

4.3 Characterization on Volta V100 using OMB

In this section, the Unified Memory support is characterized by both MVAPICH2-GDR and OpenMPI using the OMB. Figure 2 shows the performance of MVAPICH2-GDR and OpenMPI. One can clearly see that MM and HH performance for both libraries are very similar, whereas DD latency is higher compared to HH and MM configurations. This is an expected outcome because OMB only allows the effective location of the UM buffer to be the Host (CPU's) memory. In this case, the MPI library treats the UM buffer simply as a Host buffer. On the other hand, pure Device buffers are treated differently, and the expected DD numbers are seen for both MPI libraries.

The numbers presented in Figure 2 contains performance numbers for all three MM, HH, and DD configurations. However, side by side comparison is provided for the DD numbers for OpenMPI and MVAPICH2-GDR in Figure 3. Obviously, MVAPICH2-GDR significantly outperforms OpenMPI for small and medium message sizes. For large messages, the intra-node performance seems to be very comparable for OpenMPI and MVAPICH2-GDR. The reason for very similar large-message performance is that fact that there is only data movement design that is applicable for this range, i.e., CUDA IPC using cudaMemcpy operations on peer-accessible GPUs.

4.4 Characterization on Intel Systems with K80 and V100 GPUs using MOMB

Figure 4 shows the UM performance characterization on K80 using the MOMB latency benchmark. As MOMB uses sender-side CUDA kernel before MPI_Send operation, it is pertinent to mention that the latency for MM is much higher compared to HH, unlike the earlier results presented in Figure 2 using the OMB latency benchmark. MVAPICH2-GDR and OpenMPI are observed to behave differently for the K80 and MOMB case; the DD latency is always lower compared to the MM latency for MVAPICH2-GDR whereas, OpenMPI seems to have inconsistent behavior for message sizes between 4K–16K bytes. This seems to be a tuning issue where the IPC designs in OpenMPI/UCX are not being applied for message sizes smaller than 256 KB for this system. This inconsistency will be reported to the OpenMPI/UCX team, so this can be further improved, and consistent (expected) performance can be achieved.

On the other hand, Volta V100 GPUs offer much better performance compared to the K-80 GPUs for the MOMB latency benchmark as shown in Figure 5. These results are presented for V100 GPUs, but the numbers were very similar for the Pascal P100 GPUs as well. Pascal results were omitted to avoid presenting redundant information. For small and medium (non-IPC) message range, MM performance is very close to DD performance. This is the complete opposite for K80 GPUs where small and medium message MM performance (Figure 4) is significantly worse compared to DD. Figure 5 clearly shows the benefit of advanced GPU-based page-fault mechanism on Volta V100 GPUs compared to the K80 GPUs.

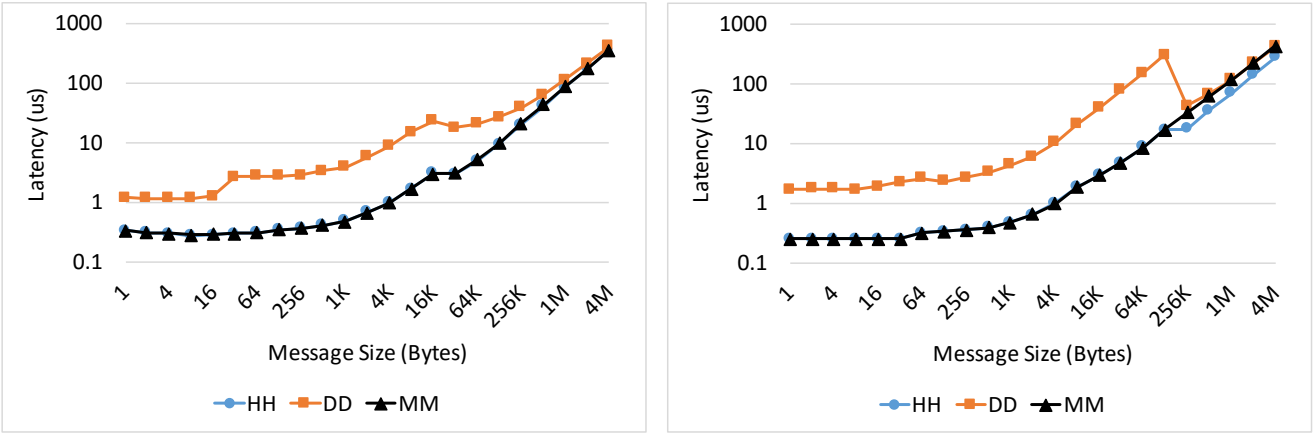
Table 1: MOMB Page Faults

GPU Architecture	Host Architecture	GPU Page Faults	CPU Page Faults
K-80	Intel X86	N/A	282800
Volta	Intel X86	31653	6550
Volta	POWER9	259	335

To verify the reasoning, MOMB was profiled using Nvidia's nvprof profiler, and the page faults are tabulated in the Table 1. The table clearly shows that the Kepler GPUs have 7.4X the number of page faults on Volta GPUs, leading to poor performance. On the other hand, for small and medium message range, the better performance trend for MM on V100 holds true for both OpenMPI and MVAPICH2-GDR. However, MVAPICH2-GDR outperforms OpenMPI in the large message range where advanced UM designs (MV2-OPT) provide benefit over basic UM support.

4.5 Characterization on POWER9 System with Volta V100 GPUs over NVLINK2 Interconnect using MOMB

As mentioned in Section 2.1, advanced hardware features on POWER9 and NVLINK2 systems potentially can bring performance improvement on unified memory access. Figure 6 reports the performance comparison between MVAPICH2-GDR, OpenMPI, and SpecturmMPI on an IBM POWER9 and NVIDIA Volta V100 GPU over NVLink2 using the MOMB latency benchmark. Unlike the observation on Intel systems presented in previous sections, data transfers of the CUDA managed buffer (*MM*) with small message sizes outperform CUDA device buffers (*DD*). We believe that this is because two reasons a) the absence of GDRCopy on the system and b) the intelligent on-demand migration of Volta GPU and hardware cache coherence with NVLink2. We also observe that, even with the advanced support like hardware cache coherence, the use of explicit copy and CUDA IPC in MVAPICH2-GDR for transferring CUDA managed buffer (*MM*) can still gain significant performance improvement over OpenMPI and SpecturmMPI for large message sizes as shown in Figure 10. As summarized in Table 1, we can see that there are significantly fewer CPU and GPU page faults on POWER9 system over NVLink2 compared to Intel systems.



(a) MVAPICH2-GDR

(b) OpenMPI (w/ UCX CUDA)

Figure 2: Unified Memory Characterization on Volta V100 with OMB

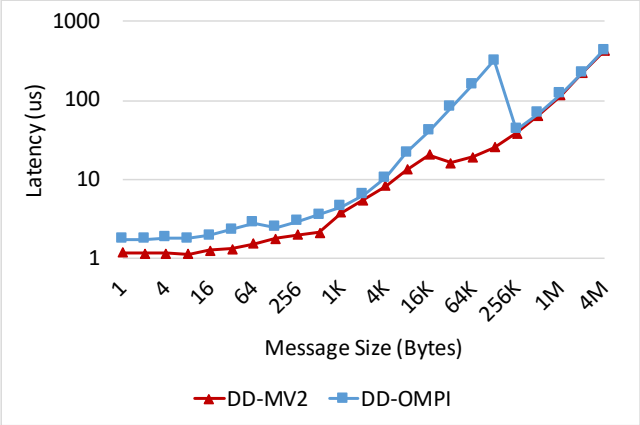


Figure 3: MVAPICH2-GDR *(MV2)* vs OpenMPI *(OMPI)*: Pure Device Buffer (DD) Performance

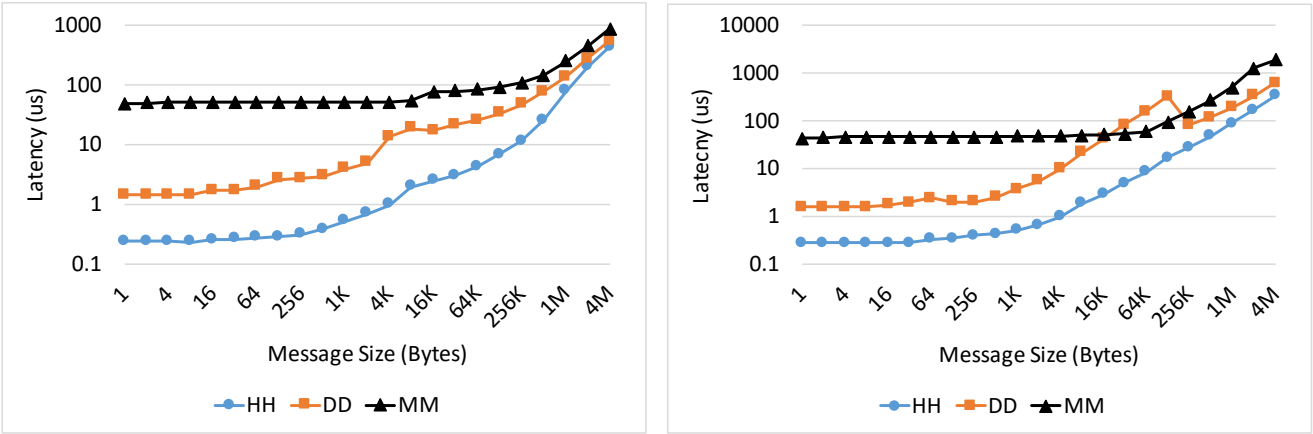
4.6 Deeper Insights and Discussion

This section illustrates the benefits of the advanced UM designs (MV2-OPT) in MVAPICH2-GDR compared to basic UM support (MV2-DEF) present in MVAPICH2-GDR. Figure 7 shows the benefits of advanced UM designs on the K80 GPUs. These advanced designs discussed in Section 3 provide high-performance solutions on top of the existing CUDA technologies such as CUDA IPC. In Figure 7 for some message sizes, inconsistent MV2-OPT vs. MV-DEF behavior is observed on the K80 GPUs. Similarly, as Figure 8 shows, the advanced UM designs does benefit the Volta V100 GPUs which already has advanced architectural features. Thus the key observation from these graphs is that advanced UM designs conceived for Kepler GPUs still holds value for Pascal and Volta GPUs and the advanced architectural features does not nullify the benefits of the advanced UM design.

The message range where advanced support in MVAPICH2-GDR seems to help starts from 32 KB onwards. This range has been separately presented in Figure 9 for three different cases: 1) MV2-OPT, 2) MV2-DEF, and 3) OMPI. MVAPICH2-GDR (MV2-OPT) outperforms OpenMPI and MVAPICH2-GDR (MV2-DEF) by 2.8× and 4.2×, respectively.

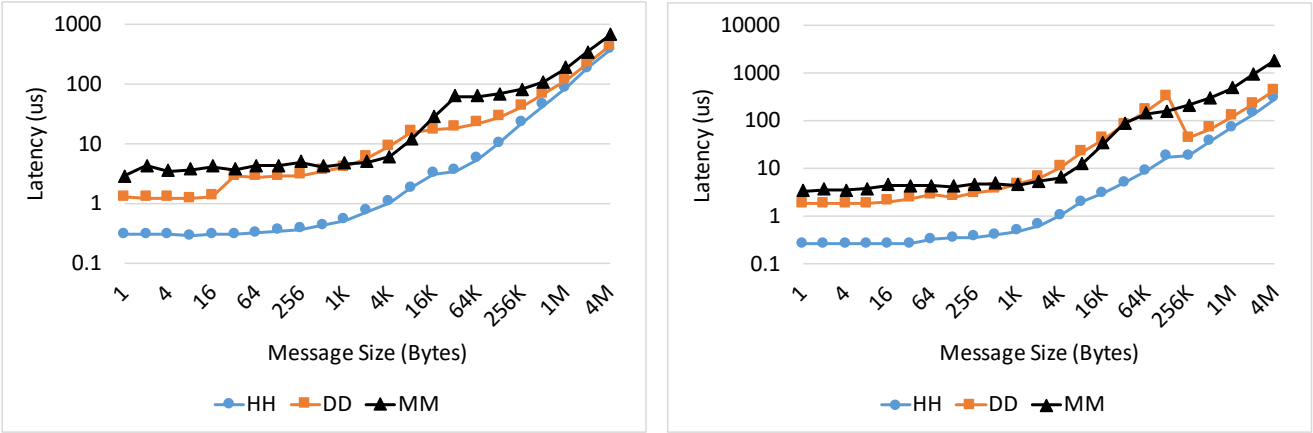
5 RELATED WORK

In the last decade, several research efforts are made by researchers to design and optimize parallel applications on GPU platforms using managed memory [3, 6, 20, 31]. While UM provides a big push for the productivity on GPUs, however, it significantly affects the runtime performance due to redundant memory transfers and page caching faults between CPU and GPU [9, 12, 14, 15, 34]. To minimize the number of redundant data transfers, Ziabari et al [37] compared zero-copy and memcpy approaches on systems with multiple GPUs and the scalable kernel execution and presented a programmer oblivious novel unified memory hierarchy. Arefyeva et al. [2] surveyed several techniques to manage GPU memory and recommended UM design to use for frequently accessed and changed data, memory mapped design for data larger than GPU memory, and divide-and-conquer design for historical data that does not get modified by GPUs. Chao et al. [16] developed a test benchmark suit and nine applications to evaluate and compare the performances of pageable memory, pinned memory, and unified memory. Using asynchronous task execution and CUDA streams, they examined the performance of concurrent GPU kernels when running with multiple tasks. Mishra et al. [19] enhanced Rodinia benchmarks to explore the impact of UM in the OpenMP context. With a



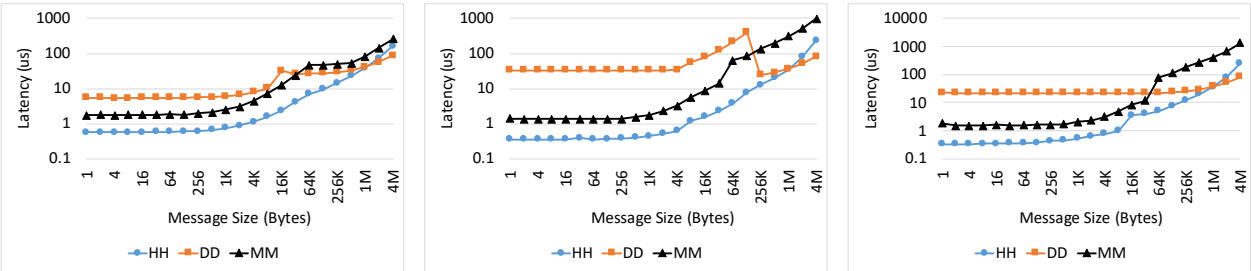
(a) MVAPICH2-GDR (b) OpenMPI (with UCX CUDA)

Figure 4: Unified Memory Characterization on Kepler K80 with MOMB



(a) MVAPICH2-GDR (b) OpenMPI (with UCX CUDA)

Figure 5: Unified Memory Characterization on Volta V100 with MOMB



(a) MVAPICH2-GDR (b) OpenMPI (with UCX CUDA) (c) SpectrumMPI

Figure 6: Unified Memory Characterization with MOMB on POWER9 and Volta over NVLink2

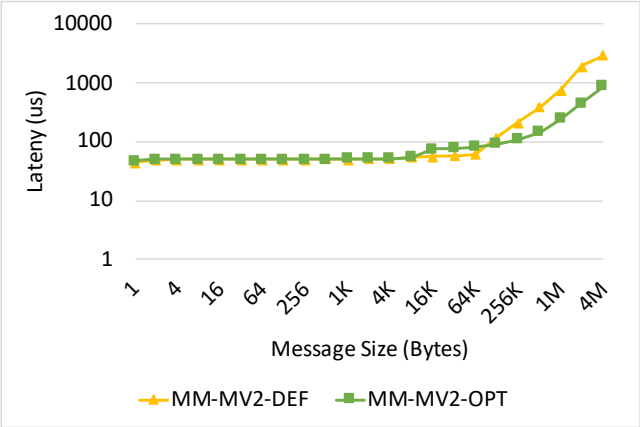


Figure 7: Gains from Advanced UM Support in MVAPICH2-GDR (MV2-OPT) on K-80 with MOMB

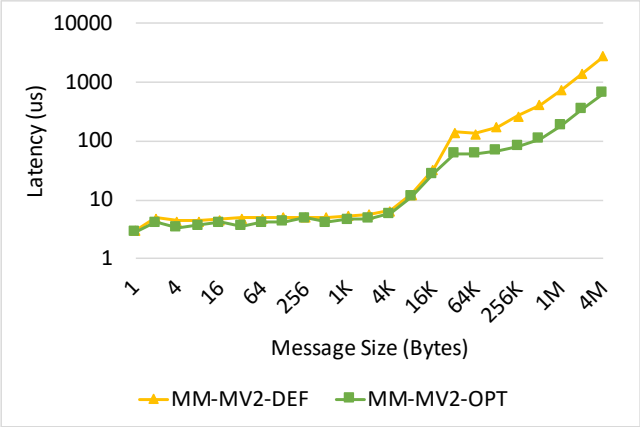


Figure 8: Gains from Advanced UM Support in MVAPICH2-GDR (MV2-OPT) on V100 with MOMB

light-weight design, the authors observed that UM delivers on par performance to normal GPU offloading at the benchmarks that do not reuse many data but incurs significant overheads at the benchmarks in which GPU memory is oversubscribed and significant amount of data is reused.

To maximize both productivity and performance potentials on GPU systems, Banerjee et al. in [7] proposed a host locality-aware design in the MVAPICH2 MPI library. A continued work through GPU locality configuration in MPI runtimes was carried by Hamidouche et al. in [11]. The authors presented a CUDA M3 framework that exploits CUDA-IPC and GDR features and delivers up to 32× and 7× performance improvements for intra-node and inter-node configurations respectively at the benchmark level, and 1.92× performance improvements for a real-world application. In a recent work, Markthub et al. in [18] presented an NVM based design that

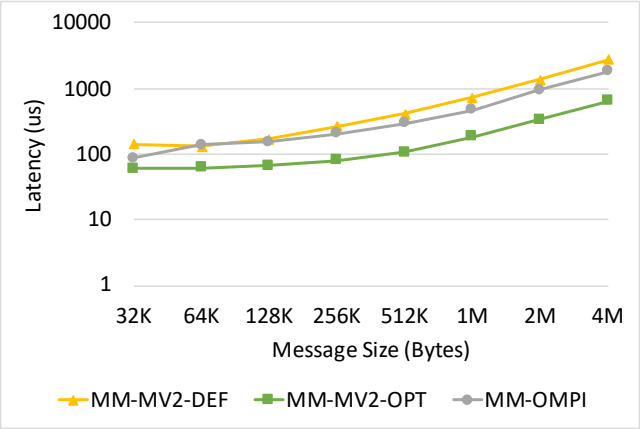


Figure 9: Large Message (IPC) Range: Benefit of Advanced UM support *(MV2-OPT)* over OpenMPI *(OMPI)* and Basic UM support *(MV2-DEF)* on V100 with MOMB

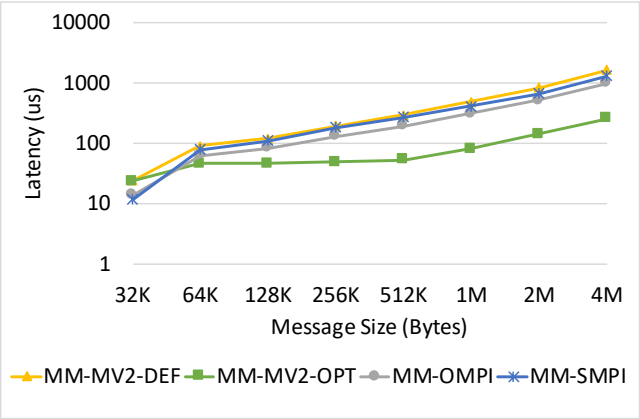


Figure 10: Large Message (IPC) Range: Benefit of Advanced UM support *(MV2-OPT)* over OpenMPI *(OMPI)*, SpectrumMPI *(SMPI)* and Basic UM support *(MV2-DEF)* on POWER9 and V100 over NVLink2

eliminates the need for manual buffer management and enables entirety of GPGPU applications to compute on terabytes of datasets residing in NVM transparently. The proposed DRAGON design leverages page-faulting mechanisms on NVIDIA GPUs and shows up to 2.3× improvements in incore-GPU and incore-host executions compared with using UM-P + POSIX IO.

6 CONCLUSION

The CUDA Unified Memory (UM) yields high productivity by providing a simpler programming paradigm. Even though it simplifies the programmer's view of CPU and GPU memory spaces, initial support for UM in the Kepler series of GPUs lacked performance, which created opportunities

for MPI runtimes to develop UM-Aware designs that offer high-performance in addition to the high-productivity. To this end, the CUDA runtime and the GPU architectures have evolved, thereby offering much better performance. In this paper, the performance of state-of-the-art MPI runtimes like MVAPICH2-GDR, OpenMPI, and Spectrum MPI on UM was characterized on three generations of GPUs including the latest Volta V100 GPUs on two different host systems (Intel and POWER9).

Our performance characterization results show that UM designs conceived during the Kepler GPU era are still valid for the Pascal and Volta era GPUs providing a valuable performance improvement. On the other hand, advanced architectural features present in the Pascal and Volta era GPUs have also contributed significantly to the performance improvement of UM.

Performance evaluations show that MVAPICH2-GDR outperforms OpenMPI and Spectrum MPI across a wide message range and benchmark/system configuration. In specific, advanced UM support branded as Managed-IPC design (MV2-OPT) in MVAPICH2-GDR offers significant performance gains for MM latency on both K80 and V100 GPUs. These observations hold true for both Intel as well as POWER9 system. MV2-OPT outperforms default MVAPICH2-GDR by 4.2× and OpenMPI by 2.8× for Intel Systems. Additionally, the DD experiments for pure device transfers also show that MVAPICH2-GDR is up to 12.6× better than OpenMPI (w/ UCX).

REFERENCES

[1] Joshua A. Anderson, Chris D. Lorenz, and A. Travesset. 2008. General Purpose Molecular Dynamics Simulations Fully Implemented on Graphics Processing Units. *J. Comput. Phys.* 227, 10 (May 2008), 5342–5359. https://doi.org/10.1016/j.jcp.2008.01.047

[2] Iya Arefyeva, David Broneske, Gabriel Campero, Marcus Pinnecke, and Gunter Saake. 2018. Memory Management Strategies in CPU/GPU Database Systems: A Survey. In *Beyond Databases, Architectures and Structures. Facing the Challenges of Data Proliferation and Growing Variety*, Stanisław Kozielski, Dariusz Mrozek, Paweł Kasprowski, Bożena Małysiak-Mrozek, and Daniel Kostrzewa (Eds.). Springer International Publishing, Cham, 128–142.

[3] A. Awan, C. Chu, H. Subramoni, X. Lu, and D. Panda. 2018. OC-DNN: Exploiting Advanced Unified Memory Capabilities in CUDA 9 and Volta GPUs for Out-of-Core DNN Training . In *25th IEEE International Conference on High Performance Computing, Data, and Analytics (HiPC)*.

[4] Ammar Ahmad Awan, Ching-Hsiang Chu, Hari Subramoni, and Dhabaleswar K. Panda. 2018. Optimized Broadcast for Deep Learning Workloads on Dense-GPU InfiniBand Clusters: MPI or NCCL?. In *EuroMPI*.

[5] Ammar Ahmad Awan, Khaled Hamidouche, Jahanzeb Maqbool Hashmi, and Dhabaleswar K. Panda. 2017. S-Caffe: Co-designing MPI Runtimes and Caffe for Scalable Deep Learning on Modern GPU Clusters. *SIGPLAN Not.* 52, 8 (Jan. 2017), 193–205. https://doi.org/10.1145/3155284.3018769

[6] K. Balhaf, M. A. Alsmirat, M. Al-Ayyoub, Y. Jararweh, and M. A. Shehab. 2017. Accelerating Levenshtein and Damerau edit distance algorithms using GPU with unified memory. In *2017 8th International Conference on Information and Communication Systems (ICICS)*.

[7] Dip Sankar Banerjee, Khaled Hamidouche, and Dhabaleswar K. Panda. 2016. Designing High Performance Communication Runtime for GPU Managed Memory: Early Experiences. In *Proceedings of the 9th Annual Workshop on General Purpose Processing Using Graphics Processing Unit (GPGPU '16)*. ACM, New York, NY, USA, 82–91. https://doi.org/10.1145/2884045.2884050

[8] COSMO. 2019. COSMO-Model. http://www.cosmo-model.org/content/model/general/default.htm Accessed: March 26, 2019.

[9] Mohammad Dashti and Alexandra Fedorova. 2017. Analyzing Memory Management Methods on Integrated CPU-GPU Systems. In *Proceedings of the 2017 ACM SIGPLAN International Symposium on Memory Management (ISMM 2017)*. ACM, New York, NY, USA, 59–69. https://doi.org/10.1145/3092255.3092256

[10] Edgar Gabriel, Graham E. Fagg, George Bosilca, Thara Angskun, Jack J. Dongarra, Jeffrey M. Squyres, Vishal Sahay, Prabhanjan Kambadur, Brian Barrett, Andrew Lumsdaine, Ralph H. Castain, David J. Daniel, Richard L. Graham, and Timothy S. Woodall. 2004. Open MPI: Goals, Concept, and Design of a Next Generation MPI Implementation. In *Proceedings, 11th European PVM/MPI Users' Group Meeting*. Budapest, Hungary, 97–104.

[11] K. Hamidouche, A. A. Awan, A. Venkatesh, and D. K. Panda. 2016. CUDA M3: Designing Efficient CUDA Managed Memory-Aware MPI by Exploiting GDR and IPC. In *2016 IEEE 23rd International Conference on High Performance Computing (HiPC)*. 52–61. https://doi.org/10.1109/HiPC.2016.016

[12] Łukasz Jarząbek and Paweł Czarnul. 2017. Performance evaluation of unified memory and dynamic parallelism for selected parallel CUDA applications. *The Journal of Supercomputing* 73, 12 (01 Dec 2017), 5378–5401. https://doi.org/10.1007/s11227-017-2091-x

[13] Jiri Kraus. 2013. An Introduction to CUDA-Aware MPI. https://devblogs.nvidia.com/introduction-cuda-aware-mpi/ Accessed: March 26, 2019.

[14] R. Landaverde, Tiansheng Zhang, A. K. Coskun, and M. Herbordt. 2014. An investigation of Unified Memory Access performance in CUDA. In *2014 IEEE High Performance Extreme Computing Conference (HPEC)*. 1–6. https://doi.org/10.1109/HPEC.2014.7040988

[15] W. Li, G. Jin, X. Cui, and S. See. 2015. An Evaluation of Unified Memory Technology on NVIDIA GPUs. In *2015 15th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing*. 1092–1098. https://doi.org/10.1109/CCGrid.2015.105

[16] Chao Liu, Janki Bhimani, and Miriam Leeser. 2017. Using High Level GPU Tasks to Explore Memory and Communications Options on Heterogeneous Platforms. In *Proceedings of the 2017 Workshop on Software Engineering Methods for Parallel and High Performance Applications (SEM4HPC '17)*. ACM, New York, NY, USA, 21–28. https://doi.org/10.1145/3085158.3086160

[17] LLNL. 2019. The ASCI UMT Benchmark Code. https://asc.llnl.gov/computing_resources/purple/archive/benchmarks/umt Accessed: March 26, 2019.

[18] Pak Markthub, Mehmet E. Belviranli, Seyong Lee, Jeffrey S. Vetter, and Satoshi Matsuoka. 2018. DRAGON: Breaking GPU Memory Capacity Limits with Direct NVM Access. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage, and Analysis (SC '18)*. IEEE Press, Piscataway, NJ, USA, Article 32, 13 pages. http://dl.acm.org/citation.cfm?id=3291656.3291699

[19] Alok Mishra, Lingda Li, Martin Kong, Hal Finkel, and Barbara Chapman. 2017. Benchmarking and Evaluating Unified Memory for OpenMP GPU Offloading. In *Proceedings of the Fourth Workshop on the LLVM Compiler Infrastructure in HPC (LLVM-HPC'17)*. ACM,

New York, NY, USA, Article 6, 10 pages. https://doi.org/10.1145/3148173.3148184

[20] J. M. Nadal-Serrano and M. Lopez-Vallejo. 2016. A Performance Study of CUDA UVM versus Manual Optimizations in a Real-World Setup: Application to a Monte Carlo Wave-Particle Event-Based Interaction Model. *IEEE Transactions on Parallel and Distributed Systems* 27, 6 (June 2016), 1579–1588. https://doi.org/10.1109/TPDS.2015.2463813

[21] Network Based Computing Laboratory. 2019. OSU Micro-benchmarks. http://mvapich.cse.ohio-state.edu/benchmarks/ Accessed: March 26, 2019.

[22] Network-Based Computing Laboratory, The Ohio State University. 2001. MVAPICH: MPI over InfiniBand, Omni-Path, Ethernet/iWARP, and RoCE. http://mvapich.cse.ohio-state.edu/ Accessed: March 26, 2019.

[23] NVIDIA. 2011. NVIDIA GPUDirect. https://developer.nvidia.com/gpudirect Accessed: March 26, 2019.

[24] NVIDIA. 2016. Whitepaper: NVIDIA Tesla P100, section 'Unified Memory'. https://images.nvidia.com/content/pdf/tesla/whitepaper/pascal-architecture-whitepaper.pdf Accessed: March 26, 2019.

[25] NVIDIA. 2017. Whitepaper: NVIDIA Tesla V100 GPU ARCHITECTURE, section 'UNIFIED MEMORY AND ADDRESS TRANSLATION SERVICES'. http://images.nvidia.com/content/volta-architecture/pdf/volta-architecture-whitepaper.pdf Accessed: March 26, 2019.

[26] NVIDIA. 2019. About CUDA | NVIDIA Developer. https://developer.nvidia.com/about-cuda Accessed: March 26, 2019.

[27] NVIDIA. 2019. Unified Memory in CUDA 6. https://devblogs.nvidia.com/unified-memory-in-cuda-6/ Accessed: March 26, 2019.

[28] S. Potluri, K. Hamidouche, A. Venkatesh, D. Bureddy, and D.K. Panda. 2013. Efficient Inter-node MPI Communication Using GPUDirect RDMA for InfiniBand Clusters with NVIDIA GPUs. In *42nd International Conference on Parallel Processing (ICPP)*. 80–89.

[29] S. Potluri, K. Hamidouche, A. Venkatesh, D. Bureddy, and D.K. Panda. 2013. Efficient Inter-node MPI Communication Using GPUDirect RDMA for InfiniBand Clusters with NVIDIA GPUs. In *42nd International Conference on Parallel Processing (ICPP)*. 80–89.

[30] S. Potluri, H. Wang, D. Bureddy, A. K. Singh, C. Rosales, and D. K. Panda. 2012. Optimizing MPI Communication on Multi-GPU Systems using CUDA Inter-Process Communication. In *Int'l Workshop on Accelerators and Hybrid Exascale Systems (AsHES), in conjunction with Int'l Parallel and Distributed Processing Symposium (IPDPS '12)*.

[31] S. T. H. Rizvi, G. Cabodi, and G. Francini. 2017. GPU-only unified ConvMM layer for neural classifiers. In *2017 4th International Conference on Control, Decision and Information Technologies (CoDIT)*. 0539–0543. https://doi.org/10.1109/CoDIT.2017.8102649

[32] Daniel Roten, Yifeng Cui, Kim B. Olsen, Steven M. Day, Kyle Withers, William H. Savran, Peng Wang, and Dawei Mu. 2016. High-frequency Nonlinear Earthquake Simulations on Petascale Heterogeneous Supercomputers. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '16)*. IEEE Press, Piscataway, NJ, USA, Article 82, 12 pages. http://dl.acm.org/citation.cfm?id=3014904.3015015

[33] Nikolay Sakharnykh. 2018. Everything You Need to Know About Unified Memory. (March 2018). http://on-demand.gputechconf.com/gtc/2018/presentation/s8430-everything-you-need-to-know-about-unified-memory.pdf

[34] Rafael Silva Santos, Danilo Medeiros Eler, and Rogério Eduardo Garcia. 2016. Performance Evaluation of Data Migration Methods Between the Host and the Device in CUDA-Based Programming. In *Information Technology: New Generations*, Shahram Latifi (Ed.). Springer International Publishing, Cham, 689–700.

[35] H. Wang, S. Potluri, D. Bureddy, C. Rosales, and D. K. Panda. 2014. GPU-Aware MPI on RDMA-Enabled Clusters: Design, Implementation and Evaluation. *IEEE Transactions on Parallel and Distributed Systems* (Oct 2014). https://doi.org/10.1109/TPDS.2013.222

[36] xBus. 2019. xBus - Overview. http://xbus.sourceforge.net Accessed: March 26, 2019.

[37] Amir Kavyan Ziabari, Yifan Sun, Yenai Ma, Dana Schaa, José L. Abellán, Rafael Ubal, John Kim, Ajay Joshi, and David Kaeli. 2016. UMH: A Hardware-Based Unified Memory Hierarchy for Systems with Multiple Discrete GPUs. *ACM Trans. Archit. Code Optim.* 13, 4, Article 35 (Dec. 2016), 25 pages. https://doi.org/10.1145/2996190